



UNIVERSITY OF HARIPUR

Department of Information & Technology
Bachelor Science in Artificial Intelligence
3rd Semester (section C)

LAB MANUAL

CC 2/14 Digital Logic Design
Miss Maria

Anwash (F23-0318) Saad (F23-0334)
Sheryar(F23-0317) Hassan (F23-0324)

<u>Lab 1: Logic Gates.....</u>	<u>2</u>
<u>Lab 2: Boolean Expression.....</u>	<u>7</u>
<u>Lab 3: POS & SOP.....</u>	<u>13</u>
<u>Lab 4: Security system.....</u>	<u>18</u>
<u>Lab 5: Half & Full Adders.....</u>	<u>23</u>
<u>Lab 6: SR flip-flop.....</u>	<u>28</u>
<u>Lab 7: T flip-flop.....</u>	<u>31</u>

Lab Manual 1

Logic Gates

1. AND Gate

Problem: Design a 2-input AND gate and verify its truth table.

- Objective: Implement a 2-input AND gate where the output is true (1) only when both inputs are true (1).
- Steps:
 1. Add 2 inputs (A and B).
 2. Add 1 AND gate.
 3. Add 1 output.
 4. Connect the two inputs to the AND gate and connect the gate's output to the output.
 5. Test the circuit by toggling the inputs and check the truth table.

Solution:



A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

2. OR Gate

Problem: Build a 2-input OR gate to simulate its logic.

- Objective: Implement a 2-input OR gate where the output is true if either or both inputs are true.

Solution:



A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

3. NOT Gate

Problem: Create a simple NOT gate circuit.

- Objective: Implement a NOT gate that inverts the input (if the input is 1, the output will be 0, and vice versa).

Solution:



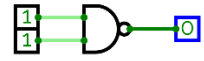
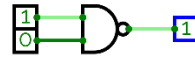
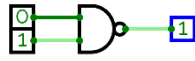
A	Output
0	1
1	0

4. NAND Gate

Problem: Design a 2-input NAND gate circuit and verify its output.

- Objective: A NAND gate gives an output of 1 unless both inputs are 1, in which case it gives 0.

Solution:



A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

5. NOR Gate

Problem: Implement a 2-input NOR gate and observe its output.

Solution:



A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

6. Exclusive OR gate.

Problem: Implement a 2-input Exclusive OR gate and observe its output.

Solution:



A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

7. Design Exclusive NOR gate.

Problem: Implement a 2-input Exclusive NOR gate and observe its output.

Solution:



A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Lab Manual 2

**Boolean Expression Design and Circuit
Implementation**

Lab Title: Boolean Expression Design and Circuit Implementation

Objective:

- To understand the basics of Boolean expressions.
 - To design and simplify Boolean expressions.
 - To implement and simulate Boolean expressions using logic gates.
 - To validate the results using the Digital Circuit Verse online simulator.
-

Experiment Overview:

In this experiment, students will learn how to represent logical statements in Boolean algebra and create the corresponding logic gate circuits to evaluate the expressions. The Boolean expressions will be simplified using Boolean algebra rules where necessary, and the final circuits will be simulated on the Circuit Verse platform.

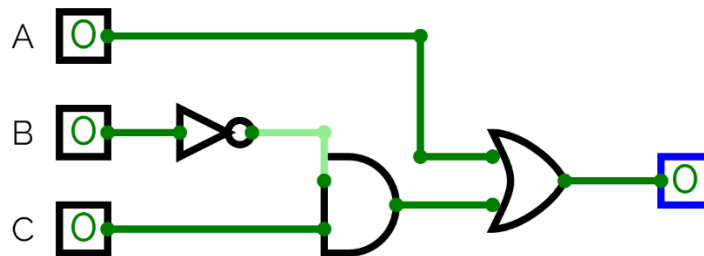
Procedure:

1. **Boolean Expression Writing:** Write down the five Boolean expressions provided below and analyze their components (AND, OR, NOT operations).
 2. **Circuit Design:** For each Boolean expression:
 - Break down the logic operation.
 - Identify the number of inputs required.
 - Construct the circuit using basic gates (AND, OR, NOT, etc.).
 3. **Simulation:**
 - Open the Circuit Verse simulator.
 - Create the digital circuit based on the given Boolean expression.
 - Test and verify the output for all possible input combinations.
 4. **Result Analysis:** Compare the simulation results with the expected truth table to verify the correctness of your circuit.
-

Expression 1: $A + B'C$

- Explanation:**

- The circuit consists of an OR gate, a NOT gate, and an AND gate.
- First, B is negated using the NOT gate.
- Then, the negated B and C are input into the AND gate.
- Finally, the output of the AND gate is ORed with A.

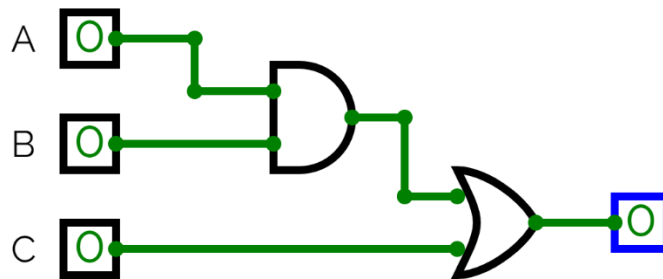
Solution:

A	B	C	Output (Y)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Expression 2: $AB + C$

- Explanation:**

- The circuit consists of an AND gate and an OR gate.
- A and B are input into the AND gate.
- The output of the AND gate is ORed with C.

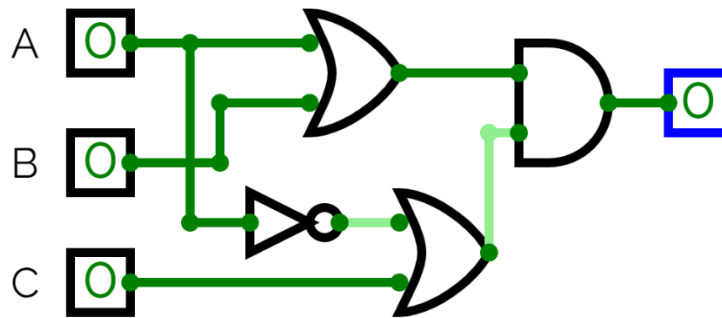
Solution:

A	B	C	Output (Y)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Expression 3: $A'B + AB'$

- Explanation:**

- The circuit uses two NOT gates, two AND gates, and an OR gate.
- A is negated and input into an AND gate with B.
- B is negated and input into an AND gate with A.
- The outputs of both AND gates are ORed together.

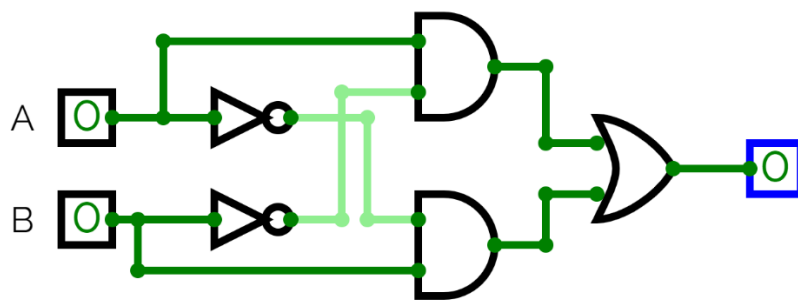
Solution:

A	B	Output (Y)
0	0	0
0	1	1
1	0	1
1	1	0

Expression 4: $(A + B)(A' + C)$

- Explanation:**

- The circuit involves two OR gates and an AND gate.
- First, create OR gates for $A+B$ and $A' + C$.
- The outputs of both OR gates are input into the final AND gate.

Solution:

A	B	C	Output (Y)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Lab Manual 3

**Designing POS and SOP Circuits
Using Circuit Verse**

Designing POS and SOP Circuits Using Circuit Verse

Objective:

To understand and implement SOP (Sum of Products) and POS (Product of Sums) forms using Circuit Verse, a cloud-based digital circuit simulator.

Implementing SOP Expression:

SOP Expression: $F(A, B, C) = A'B'C + AB'C' + ABC$

Steps to Create the SOP Circuit:

1. Understanding the SOP expression:

- Break the expression into minterms. In this case:

Minterms: $A'B'C$, $AB'C'$, ABC

2. Inserting Inputs:

- From the toolbar on the left, drag three inputs (A, B, and C) to the work area.

3. NOT Gates:

- For the inverted terms (A' , B' , C'), drag three NOT gates from the toolbar.
- Connect A to A' using a NOT gate. Repeat this for B' and C' .

4. AND Gates:

- For each minterm, use AND gates:
 - Connect A' , B' , and C to an AND gate.
 - Connect A, B' , and C' to another AND gate.
 - Connect A, B, and C to a third AND gate.

5. OR Gate:

- To combine the outputs of the AND gates, drag an OR gate and connect the outputs of all three AND gates to the OR gate.

6. Output:

- Connect the output of the OR gate to an output label (F).

7. Simulate:

- Test different inputs for A, B, and C to verify the output matches the SOP expression.
-

Implementing POS Expression:

POS Expression: $F(A, B, C) = (A+B+C')(A'+B+C)(A'+B'+C')$

Steps to Create the POS Circuit:**1. Understanding the POS expression:**

- Break the expression into maxterms. In this case:

Maxterms: $(A+B+C')$, $(A'+B+C)$, $(A'+B'+C')$

2. Inserting Inputs:

- Use the same inputs (A, B, and C) as the SOP design.

3. NOT Gates:

- For the inverted terms (A' , B' , C'), use NOT gates as done earlier.

4. OR Gates:

- For each maxterm, use OR gates:
 - Connect A, B, and C' to an OR gate.
 - Connect A' , B, and C to another OR gate.
 - Connect A' , B' , and C' to a third OR gate.

5. AND Gate:

- To combine the outputs of the OR gates, drag an AND gate and connect the outputs of all three OR gates to the AND gate.

6. Output:

- Connect the output of the AND gate to an output label (F).

7. Simulate:

- Test the inputs for A, B, and C to verify the output matches the POS expression.

Testing:

1. Modify the input values for A, B, and C and observe the changes in the output.
 2. Verify that the output values match the expected truth table for both SOP and POS forms.
-

Documenting Observations:

- After completing the circuit design, students should fill out the following table in their report:

Input A	Input B	Input C	SOP Output	POS Output
0	0	0	0	1
0	0	1	1	1
0	1	0	0	1
1	1	1	1	0

SOP Expression:

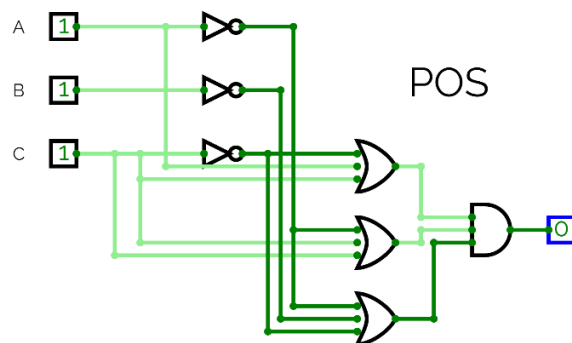
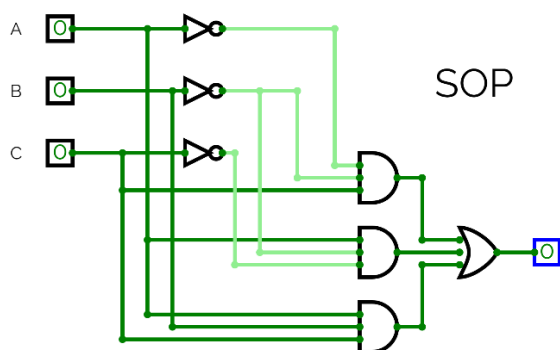
$$F(A, B, C) = A'B'C + AB'C' + ABC$$

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

POS Expression:

$$F(A, B, C) = (A+B+C')(A'+B+C)(A'+B'+C')$$

A	B	C	Output
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0



Lab Manual 4

DESIGNING A BASIC HOMESECURITY SYSTEM

DESIGNING A BASIC HOME SECURITY SYSTEM

Objective:

Design a security system that triggers an alarm (represented by an LED bulb) if certain conditions are met, such as doors or windows being open, or movement detected inside the house. The alarm should activate only when the system is "armed."

Learning Outcomes:

By completing this lab, students should be able to:

1. Design circuits using basic logic gates based on real-world conditions.
2. Understand the role of conditional logic in security systems.
3. Visualize outputs using LED bulbs to confirm circuit functionality.

Required Components in Circuit Verse:

- Switches (representing sensors for doors, windows, and motion)
 - AND, OR, NOT gates
 - LED bulbs (representing alarms)
-

Scenario

Problem Statement:

Design a circuit for a home security system with the following conditions:

1. The system is activated by a switch labelled **Armed**.
2. There are two door sensors (Door 1 and Door 2) and one window sensor (Window 1).
3. A motion sensor detects movement inside the house.
4. The alarm (bulb) should turn on if:
 - The system is **Armed**, and any **door** or **window** is open, or
 - The system is **Armed**, and **motion** is detected inside the house.

Solution Steps

1. Define Inputs and Outputs:

- **Armed (A):** Switch that activates the security system.
- **Door 1 (D1):** Switch for the first door sensor (ON if the door is open).
- **Door 2 (D2):** Switch for the second door sensor (ON if the door is open).
- **Window 1 (W1):** Switch for the window sensor (ON if the window is open).
- **Motion (M):** Switch for the motion sensor (ON if motion is detected).
- **Alarm (AL):** LED bulb that lights up if the alarm condition is met.

2. Set Up Conditions:

- The alarm should activate if:
 - The system is Armed when at least one of the doors or the window is open
 - The system is Armed when motion is detected.
- Boolean Expression for the alarm (AL): $AL = A \cdot ((D1 + D2 + W1) + M)$
- Here:
 - **A** is "ANDed" with the rest of the conditions, meaning the alarm only activates if the system is Armed.
 - $(D1 + D2 + W1)$ is an OR operation that checks if any door or window is open.
 - **M** is included as a separate condition in case there is motion.

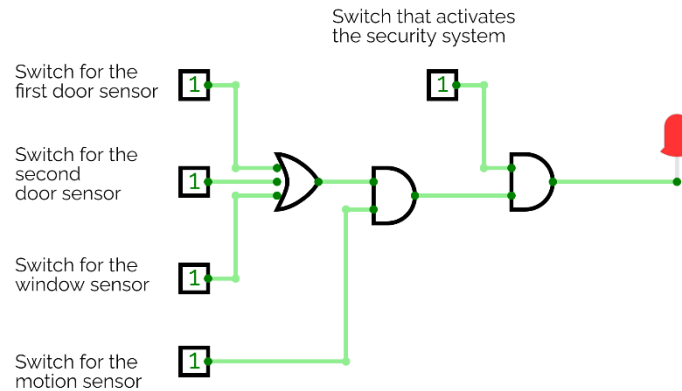
3. Implement the Circuit in Circuit Verse:

- Open Circuit Verse and create a new circuit.
- Add switches labelled **A**, **D1**, **D2**, **W1**, and **M**.
- Add an LED bulb for **AL** to represent the alarm.

4. Construct the Logic Gate Circuit:

- **Step 1:** Create the condition $(D1 + D2 + W1)$ using an OR gate.
 - Connect **D1**, **D2**, and **W1** to a 3-input OR gate.
- **Step 2:** Add an OR gate to combine the output of the first OR gate with **M**.
 - This creates the expression $(D1 + D2 + W1) + M$.
- **Step 3:** Use an AND gate to combine **A** with the output of the OR gate.
 - This step ensures that the alarm only activates if the system is Armed.

- **Step 4:** Connect the output of the final AND gate to the LED bulb (AL).



5. Test the Circuit:

- **Case 1:** Set **A = 1** (system Armed) and set **D1 = 1** (Door 1 open). The bulb should light up.
- **Case 2:** Set **A = 1**, **D2 = 1** (Door 2 open). The bulb should light up.
- **Case 3:** Set **A = 1**, **W1 = 1** (Window 1 open). The bulb should light up.
- **Case 4:** Set **A = 1**, **M = 1** (motion detected). The bulb should light up.
- **Case 5:** Set **A = 0** (system disarmed), and any of **D1**, **D2**, **W1**, or **M** to 1. The bulb should remain OFF, as the system is disarmed.

6. Truth Table Verification:

Armed (A)	Door 1 (D1)	Door 2 (D2)	Window 1 (W1)	Motion (M)	Alarm (AL)
0	0	0	0	0	0
0	1	0	0	0	0
1	1	0	0	0	1
1	0	1	0	0	1
1	0	0	1	0	1
1	0	0	0	1	1
1	1	1	1	1	1

7. **Conclusion** The circuit functions correctly as a home security system: the alarm (AL) only lights up if the system is Armed and detects a security breach (an open door, window, or motion inside). This exercise demonstrates how logic gates can be used to create conditional systems in real-world applications.
-

Lab Manual 5

Half and Full Adders

Combinational Logic Circuits - Half and Full Adders

Scenario

You have been hired as a junior digital circuit designer by a tech company that is working on the development of an efficient, low-power calculator circuit for a line of basic computing devices. Your task is to design and test the most fundamental building blocks for binary addition circuits: the half adder and the full adder. These circuits are essential in larger arithmetic circuits that perform binary addition on multiple bits.

As a part of your training, your supervisor has assigned you a lab project to create and test half and full adders using basic logic gates.

Lab Objectives

- **Understand** the theory and operation of combinational circuits, specifically focusing on half and full adders.
 - **Design** the circuits for a half adder and a full adder using basic logic gates (AND, OR, XOR).
 - **Implement** the designed circuits using Circuit verse.
 - **Verify** the outputs against the expected results using truth tables.
 - **Analyze** the results and note any discrepancies in output, if any.
 - **Report** the functionality and potential applications of these circuits in larger digital systems.
-

Problem Statement

Task: You are tasked with designing, implementing, and testing a half adder and a full adder circuit. The lab will require you to understand the binary addition process, create the circuit diagrams, implement them using circuit verse, and validate the functionality by comparing the observed outputs with the theoretical truth tables.

Requirements:

1. Half Adder Design and Implementation

- Design a half adder circuit that can add two single-bit binary inputs (A and B).
- Use basic gates to build the half adder and identify the outputs: Sum (S) and Carry (C).
- Implement the circuit using a digital simulation tool Circuit verse.
- Test the circuit by applying all possible combinations of inputs (00, 01, 10, 11) and record the outputs.

- Compare your results with the theoretical truth table of the half adder to verify its accuracy.

2. Full Adder Design and Implementation

- Design a full adder circuit that can add three single-bit binary inputs: two significant bits (A and B) and a carry-in (Cin).
- Use a combination of basic gates (AND, OR, XOR) to derive the outputs: Sum (S) and Carry-out (Cout).
- Implement this circuit on a simulation tool.
- Apply all possible input combinations (000 to 111) and record the observed outputs.
- Compare the observed outputs with the truth table of the full adder to confirm the circuit's operation.

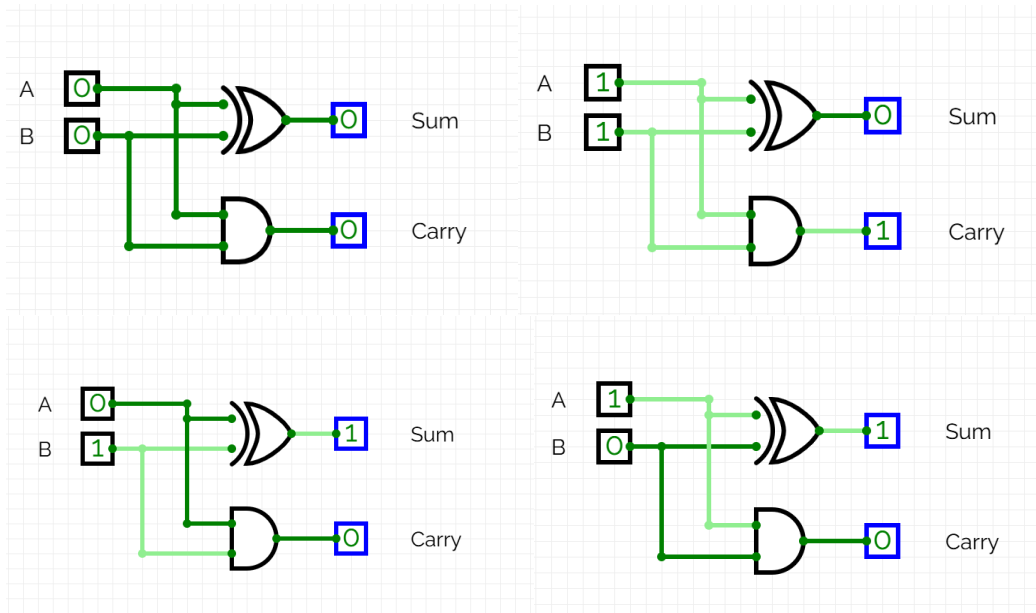
Expected Outcomes:

- Successful design and implementation of both half and full adders.
- Accurate measurement and recording of results.
- Correct output that matches the theoretical truth tables for each circuit.

By completing this lab, you will gain practical skills in designing combinational logic circuits, understanding binary arithmetic, and applying logic gates to solve real-world problems in digital systems.

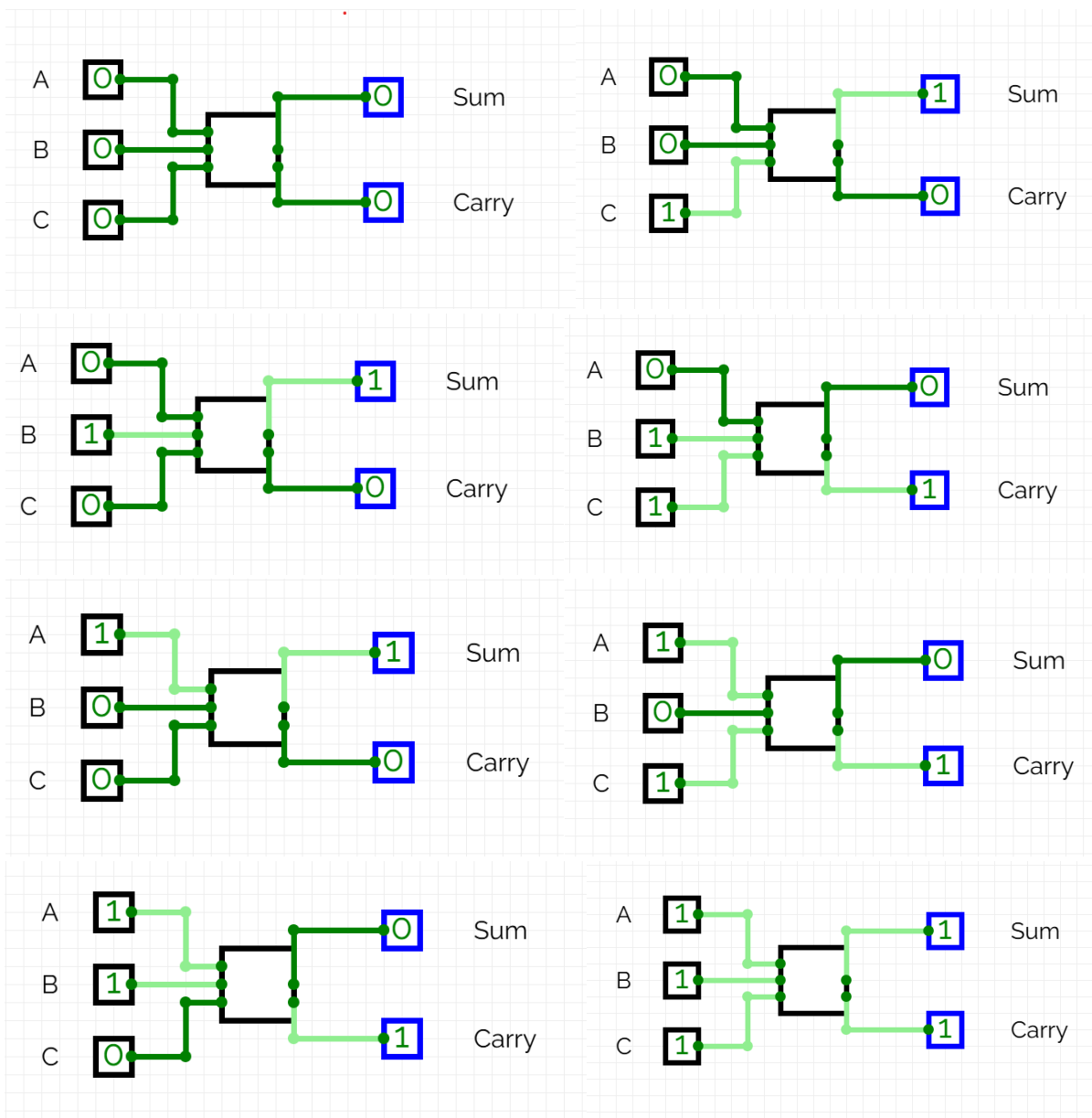
Half Adder Design and Implementation

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Full Adder Design and Implementation

A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Lab Manual 6

SR Flip-Flop

Lab Manual for SR Flip-Flop Using Circuit Verse

Experiment Title: Design and Simulation of SR Flip-Flop using Circuit Verse

Objective:

1. To design and simulate SR Flip-Flop using basic logic gates in Circuit Verse.
2. To analyze the behaviour of the SR Flip-Flop and verify its truth table.

Procedure:

1. Design the Circuit:

- Add two **NAND gates** from the toolbar.
- Connect the **S (Set)** input to one NAND gate.
- Connect the **R (Reset)** input to the other NAND gate.
- Cross-connect the outputs of the NAND gates to the other gate's second input to form feedback loops.
- Label the outputs as Q and Q'.

2. Simulation:

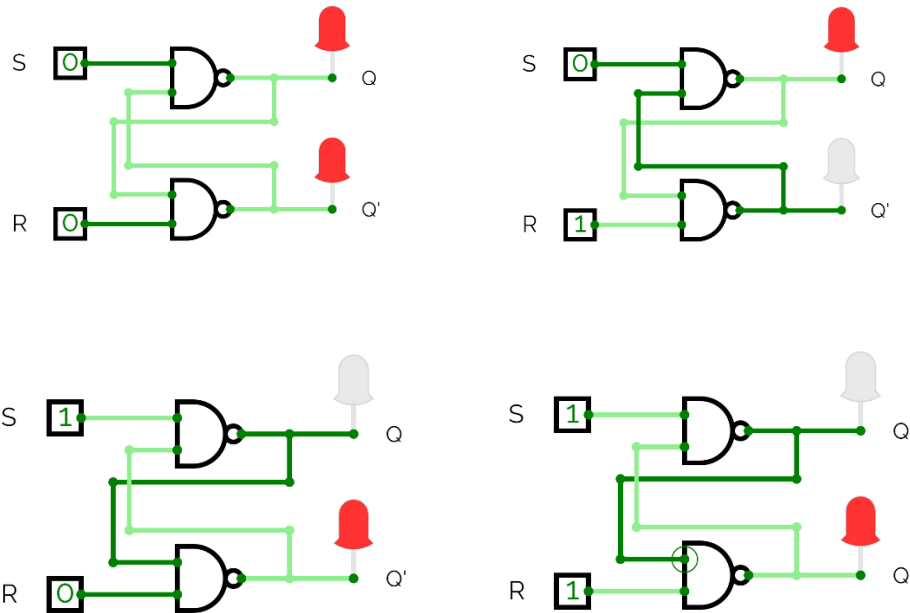
- Add **Switches** for the inputs S and R.
- Add **LEDs** to observe the outputs Q and Q'.
- Simulate the circuit by toggling the switches and observing the outputs.

3. Record Observations:

- Verify the output for all input combinations: (S, R) = (0, 0), (0, 1), (1, 0), (1, 1).
-

Design and Simulation of SR Flip-Flop

S	R	Q	Q'
0	0	1	1
0	1	1	0
1	0	0	1
1	1	0	1



Invalid at (S=0,R=0) and holding or showing previous output at (S=1,R=1). In this case it was (0,1), so it shows same output (0,1), on (S=1,R=1).

Lab Manual 7

T Flip-Flop

Toggling LEDs Using T Flip-Flop

Objective

To demonstrate the operation of a T flip-flop by controlling two LED bulbs. The LEDs will toggle based on the T input and clock signal, showing alternating ON/OFF effects.

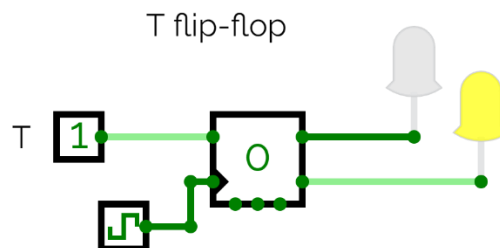
Theory:

A T-Flip Flop (Toggle Flip Flop) is a type of sequential logic circuit and a variant of the JK Flip Flop. It has a single input, typically labeled as "T," and toggles its output state (from 0 to 1 or from 1 to 0) on every clock pulse when the input is high ($T=1$). If the input is low ($T=0$), the output remains unchanged.

C	T	Q_{n+1}
trigger	0	Q_n
trigger	1	Q_n'

Components Required:

- 1 Input (T): To control toggling.
- 1 Clock Signal: To trigger the flip-flop.
- 2 LED bulbs connected to the flip-flop's outputs.



Operation:

- Set $T = 1$ to enable toggling of the LEDs with each clock pulse.
- Observe the LEDs alternate between ON/OFF states.
- Set $T = 0$ to hold the current state of the LEDs (no toggling).